



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Hálózati Rendszerek és Szolgáltatások Tanszék

Posztkvantum-kriptográfia alkalmazása az IoT rendszerek biztonságának megerősítésére

SZAKDOLGOZAT

Készítette
Király Róbert

Konzulens
Gódor Győző

2026. május 29.

Tartalomjegyzék

Kivonat	i
Abstract	ii
1. Bevezetés	1
1.1. A kriptográfia és az információbiztonság rövid története	1
1.2. Kvantumszámítógépek	3
1.2.1. A kvantumszámítás működési elvei	3
1.2.2. A kvantumszámítógépek előnyei és lehetőségei	4
1.2.3. Veszélyek a jelenlegi kriptográfiai rendszerekre	4
1.3. A dolgok internete	5
1.3.1. Az IoT kialakulása és fejlődése	5
1.3.2. IoT eszközök jellemzői és korlátai	6
1.3.3. IoT biztonsági kihívások	6
1.4. Posztkvantum-kriptográfia és IoT — a kapcsolat	7
1.5. A dolgozat céljai és felépítése	8
2. Posztkvantum-kriptográfiai protokollok	9
2.1. Posztkvantum-kriptográfia matematikai alapjai	9
2.1.1. A rácselmélet alapjai	9
2.1.2. Az LWE probléma és annak moduláris változata	10
2.2. CRYSTALS-Kyber protokoll	10
2.2.1. Paraméterek és biztonsági szintek	10
2.2.2. Kulcscsere és kulcsencapsulation	11
2.2.2.1. Kulcsgenerálás (KeyGen)	11
2.2.2.2. Üzenet encapsulation (Encaps)	11
2.2.2.3. Üzenet decapsulation (Decaps)	12
2.2.3. Biztonsági tulajdonságok és hardver gyorsulás	12
2.3. BIKE (Bit Flipping Key Encapsulation) protokoll	12
2.3.1. Kód-alapú titkosítás alapelvei	12
2.3.2. MDPC kódok és az BIKE paraméterek	13
2.3.3. BIKE kulcsgenerálás és encapsulation	13
2.3.3.1. Kulcsgenerálás (KeyGen)	13
2.3.3.2. Üzenet encapsulation (Encaps)	14
2.3.3.3. Üzenet decapsulation (Decaps) és bit-flipping dekódolás . .	14
2.3.4. Biztonsági tulajdonságok	14
2.4. Oldalsávós támadások és biztonsági implementáció	15
2.4.1. Időzítési támadások (Timing Attacks)	15
2.4.2. Energiafogyasztási támadások (Power Analysis)	15
2.4.3. Hibabeviteli támadások (Fault Injection)	15
2.5. IoT alkalmazások és integráció	16

2.5.1.	Kyber IoT implementáció	16
2.5.2.	BIKE IoT implementáció	16
2.5.3.	Hibrid megközelítés az IoT-ben	16
2.5.4.	Memória- és tárolási követelmények	17
2.6.	Kyber és BIKE összehasonlítása	17
2.6.1.	Biztonsági megfontolások	17
2.6.2.	Teljesítmény- és méret-jellemzők	17
2.6.3.	Gyakorlati alkalmazások	17
2.6.4.	Hibrid megközelítés	18
3.	Mérési környezet	19
3.1.	Hardverkörnyezet	19
3.1.1.	Tesztplatform specifikációja	19
3.1.2.	Hálózati topológia	19
3.1.3.	Energiaellátás és mérőeszközök	19
3.2.	Szoftverkörnyezet	19
3.2.1.	Operációs rendszer és alapkönyvtárak	19
3.2.2.	Kriptográfiai könyvtárak	19
3.2.3.	MQTT infrastruktúra	19
3.2.4.	Konténerizáció és reprodukálhatóság	19
3.3.	A mérési környezet összefoglalása	19
4.	Mérés metodológia	20
4.1.	Mosquitto alapú MQTT laboratóriumi környezet kialakítása posztkvantum kriptográfiai támogatással	20
4.1.1.	Bevezetés	20
4.1.2.	A labor célja és architektúrája	20
4.1.3.	Operációs rendszer és alapsomagok telepítése	21
4.1.4.	Docker környezet kialakítása	22
4.1.5.	Az Open Quantum Safe projekt komponensei	22
4.1.6.	liboqs fordítása és telepítése	23
4.1.7.	OpenSSL 3.3 fordítása	23
4.1.8.	OQS Provider fordítása	24
4.1.9.	Környezeti változók konfigurálása	24
4.1.10.	Az OpenSSL működésének ellenőrzése	24
4.1.11.	Mosquitto MQTT broker telepítése	24
4.1.12.	TLS tanúsítványok létrehozása	24
4.1.13.	Mosquitto TLS konfiguráció	25
4.1.14.	A broker indítása és tesztelése	25
4.1.15.	MQTT kommunikáció ellenőrzése	25
4.1.16.	Összegzés	26
5.	Mérések	27
5.1.	Mérési forgatókönyvek	27
5.1.1.	Alap kriptográfiai műveletek mérése	27
5.1.2.	TLS kézfogás (handshake) mérése	27
5.1.3.	Terheléses mérés	27
5.2.	Mérési módszer és adatgyűjtés	27
5.2.1.	Időmérés megvalósítása	27
5.2.2.	Memóriahasználat mérése	27
5.2.3.	Energiafogyasztás mérése	27

5.3.	Mérési eredmények	27
5.3.1.	Kyber-512 eredmények	27
5.3.2.	Kyber-768 és Kyber-1024 eredmények	27
5.3.3.	BIKE-L1 eredmények	27
5.3.4.	BIKE-L3 és BIKE-L5 eredmények	27
5.3.5.	TLS kézfogás összesítő adatok	27
5.4.	A mérések összefoglalása	27
6.	Eredmények kiértékelése	28
6.1.	Teljesítmény összehasonlítása	28
6.1.1.	Futási idő összehasonlítása	28
6.1.2.	Memóriaigény összehasonlítása	28
6.1.3.	Kulcs- és kapszulaméret összehasonlítása	28
6.1.4.	TLS kézfogás teljesítménye	28
6.2.	Biztonsági szempontú értékelés	28
6.2.1.	Biztonsági szintek és NIST kategóriák	28
6.2.2.	Ismert kriptanalitikai eredmények	28
6.2.3.	Oldalsáv-támadásokkal szembeni ellenállás	28
6.3.	Alkalmazhatósági elemzés	28
6.3.1.	Erőforrás-korlátozott eszközökre vonatkozó következtetések	28
6.3.2.	Protokoll-szintű kihívások	28
6.3.3.	Kyber és BIKE összehasonlító értékelése	28
6.4.	Az eredmények összefoglalása	28
7.	Összefoglalás és javaslatok	29
7.1.	A dolgozat eredményeinek összefoglalása	29
7.1.1.	Elméleti eredmények	29
7.1.2.	Mérési eredmények összefoglalása	29
7.2.	Javaslatok IoT rendszerek PQC migrációjához	29
7.2.1.	Algoritmusválasztási útmutató	29
7.2.2.	Hibrid megközelítés alkalmazása	29
7.2.3.	Cryptographic agility tervezési minták	29
7.2.4.	Protokoll- és infrastruktúra-ajánlások	29
7.3.	A kutatás korlátai	29
7.4.	Jövőbeli kutatási irányok	29
7.4.1.	Kibővített hardverplatform-vizsgálat	29
7.4.2.	Teljes end-to-end IoT rendszer mérése	29
7.4.3.	Post-quantum PKI és tanúsítványkezelés	29
7.4.4.	Energiahatékonyság optimalizálása	29
	Köszönetnyilvánítás	30
	Irodalomjegyzék	31
	Függelék	32
F.1.	Nyilatkozat generatív mesterséges intelligencia alkalmazásáról	32

HALLGATÓI NYILATKOZAT

Alulírott *Király Róbert*, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Budapest, 2026. május 29.

Király Róbert
hallgató

Kivonat

Jelen dokumentum egy diplomaterv sablon, amely formai keretet ad a BME Villamosmérnöki és Informatikai Karán végző hallgatók által elkészítendő szakdolgozatnak és diplomatervnek. A sablon használata opcionális. Ez a sablon $\text{\LaTeX}$ alapú, a *TeXLive* $\text{\TeX}$ -implementációval és a PDF- $\text{\LaTeX}$ fordítóval működőképes.

Abstract

This document is a L^AT_EX-based skeleton for BSc/MSc theses of students at the Electrical Engineering and Informatics Faculty, Budapest University of Technology and Economics. The usage of this skeleton is optional. It has been tested with the *TeXLive* T_EX implementation, and it requires the PDF-L^AT_EX compiler.

1. fejezet

Bevezetés

Az emberiség kommunikációjának védelme iránti igény egyidős a civilizációval. Az írott információ titkosítása évezredek óta folyamatosan fejlődő diszciplína, amelyet minden kor-szak saját technológiai szintjéhez igazított. A 20. század végén lezajlott digitális forradalom azonban minőségileg új helyzetet teremtett: a kriptográfia többé nem pusztán katonai vagy diplomáciai eszköz, hanem a globális digitális infrastruktúra alapköve. Bankrendszerek, egészségügyi nyilvántartások, ipari vezérlőrendszerek és hétköznapi kommunikáció egyaránt a nyilvános kulcsú kriptográfia védelmére támaszkodik. E rendszerek biztonsága matematikai problémák számítási nehézségére épül — egy feltevésre, amelynek érvényessége a közeljövőben komolyan megkérdőjeleződhet.

Párhuzamosan a digitális forradalom kibontakozásával egy másik technológiai változás is lezajlott: a dolgok internete (Internet of Things, IoT) robbanásszerű terjedése. Beágyazott processzorok, érzékelők és hálózati kapcsolat ma már nem csupán számítógépekben, hanem ipari gépekben, orvosi implantátumokban, okos otthonokban és kritikus infrastruktúrában is megtalálható. A becslések szerint 2030-ra több mint 25 milliárd hálózatra kötött eszköz lesz forgalomban [7], amelyek mindegyike érzékeny adatokat generál, tárol és továbbít, miközben erőforrásaikban — számítási teljesítményben, memóriában és energiában — erősen korlátozottak.

A két tendencia találkozásánál egy harmadik, egyre fenyegetőbb fejlemény jelenik meg: a kvantumszámítógépek fejlődése. A kvantummechanika elvein alapuló számítási paradigma alkalmas arra, hogy a ma használt nyilvános kulcsú kriptográfiai rendszerek matematikai alapjait polinomiális időn belül megoldja. Ez azt jelenti, hogy egy kellően erős kvantumszámítógép megjelenésével a jelenlegi titkosítási infrastruktúra teljes egészében sérülékennyé válik — nem csupán a jövőbeli kommunikáció, hanem a ma titkosított és elmentett forgalom is, amelyet a jövőben visszafejtenek.

A dolgozat célja, hogy megvizsgálja, hogy hogyan lehet posztkvantum-kriptográfiai (PQC) megoldásokat — konkrétan a CRYSTALS-Kyber és a BIKE kulcskapszulázó algoritmusokat — hatékonyan alkalmazni erőforrás-korlátozott IoT környezetekben. A bevezető fejezet az elméleti háttérrel tárgyalja: bemutatja a kriptográfia fejlődésének történetét, részletezi a kvantumszámítógépek működési elvét, előnyeit és veszélyeit, majd ismerteti az IoT ökoszisztéma sajátosságait és biztonsági kihívásait, végül bemutatja a PQC és IoT területek kapcsolatát.

1.1. A kriptográfia és az információbiztonság rövid története

A titkosítás legkorábbi ismert példái az ókori Egyiptomból származnak, ahol Kr. e. 1900 körül módosított hieroglifákat alkalmaztak a szöveg értelmének elrejtésére. Az ókori Görögországban a szkütalé nevű hengerrel transzpozíciós titkosítást végeztek, míg Julius Caesar

a nevét viselő Caesar-rejtjellel titkosította katonai üzeneteit: minden betűt az ábécé harmadik következő betűjével helyettesített. Ezek az ún. klasszikus rejtjelek helyettesítésen vagy transzpozíción alapultak, és a kriptóanalízis fejlődésével rendszeresen feltörhetővé váltak.

A középkorban a polialfabetikus rejtjelezés megjelenése (Vigenère-rejtjel, 1553) egy lépéssel előrébb vitte a titkosítás nehézségét, ám a Kasiski-teszt (1863) és a Friedman-koincidenziindex (1920-as évek) módszereivel ez a megközelítés is feltörhetőnek bizonyult. A klasszikus kriptográfia lényegi korlátja az volt, hogy minden rendszer biztonsága a kulcs titkosságán alapult, miközben a kulcsot valahogyan biztonságosan el kellett juttatni a kommunikáló felekhez — ez az ún. kulcsterjesztési probléma, amely évszázadokig megoldhatatlannak tűnt.

A modern kriptográfia alapjait a 20. század első felében fektették le. A második világháborúban az Enigma-gép és az annak feltörésére irányuló bletchley-i erőfeszítések döntő szerepet játszottak a konfliktus kimenetelében, és egyben megalapozták a számítástechnika és az algoritmikus kriptográfia fejlődését. Alan Turing, Gordon Welchman és munkatársaik az elektromechanikus Bombe gépek segítségével automatizálták az Enigma-kulcsok feltörését, és ezzel a kriptanalízist ipari méretűvé tették. Claude Shannon 1949-es munkája, a *Communication Theory of Secrecy Systems* matematikai alapokra helyezte az információelmélet és a kriptográfia kapcsolatát: bevezette az entrópia, a tökéletes titkosság és a biztonság formális fogalmait [12]. Shannon megmutatta, hogy tökéletes titkosság egyetlen rendszertől, az egyszer használatos rejtjeltől (one-time pad) várható el, amely azonban kulcskezelési okokból nem praktikusan alkalmazható.

Az 1970-es évek hoztak igazi forradalmat. 1973-ban az USA Nemzeti Szabványügyi Intézete (NBS, ma NIST) meghirdette az egységes adattitkosítási szabvány pályázatát, amelyből 1977-ben a DES (Data Encryption Standard) lett. A DES egy 64 bites blokkméretű, 56 bites kulcsú szimmetrikus rejtjel, amely az IBM Lucifer algoritmusán alapult, és amelynek tervezésébe az NSA is beleszólt. A DES évtizedekig ipari szabvány maradt, ám 1998-ban az EFF (Electronic Frontier Foundation) *Deep Crack* nevű dedikált hardverével 56 óra alatt visszafejtett egy DES-titkosított üzenetet, egyértelműen bizonyítva az algoritmus elavultságát. Utódja, a 2001-ben szabványosított AES (Advanced Encryption Standard) — Vincent Rijmen és Joan Daemen Rijndael algoritmusán alapulva — máig az egyik legerősebb szimmetrikus titkosítási algoritmus.

Az igazi paradigmaváltást azonban a nyilvános kulcsú kriptográfia 1976-os bevezetése jelentette. Whitfield Diffie és Martin Hellman *New Directions in Cryptography* című munkájukban [5] egy merőben új elvet fogalmaztak meg: a titkos kulcs megosztása nélkül is lehetséges biztonságos kommunikáció létrehozása. Ez az ún. aszimmetrikus kriptográfia alapelve: egy nyilvános kulccsal bárki titkosíthat, de visszafejteni csak a megfelelő privát kulcs birtokosa képes. A Diffie–Hellman kulcscsere a diszkrét logaritmus problémán alapul, amelynek megoldása a csoport megfelelő megválasztásával praktikusan kivitelezhetetlennek tűnt.

Ezt az ötletet rövidesen Ron Rivest, Adi Shamir és Leonard Adleman ültette át a gyakorlatba az RSA algoritmus formájában (1977, közzétéve 1978-ban), amelynek biztonsága a nagy számok faktorizálásának nehézségén alapul. Az 1980-as és 1990-es évtizedben Neal Koblitz és Victor Miller egymástól függetlenül javasolta az elliptikus görbe kriptográfia (ECC) alkalmazását, ahol a csoportként elliptikus görbék pontjait alkalmazzák. Az ECC kisebb kulcsmérettel nyújt azonos biztonsági szintet az RSA-hoz képest, ami mobil és beágyazott alkalmazásokban komoly előnnyé vált.

A 2000-es évektől a kriptográfia az internet mindennapi infrastruktúrájává vált. A TLS (Transport Layer Security) protokoll révén az RSA és az ECC az HTTPS, a biztonságos e-mail és a VPN alapszövevényként működik. A nyilvános kulcsú infrastruktúra (PKI) tanúsítványok millióit gondozza, és az internet kommunikáció biztonságának gerincét al-

kotja. Ez a globális, komplex infrastruktúra áll most szemben a kvantumszámítástechnika által felvetett kihívással.

1.2. Kvantumszámítógépek

A kvantumszámítógépek nem egyszerűen gyorsabb klasszikus számítógépek: egy alapvetően más számítási paradigmát képviselnek, amely a kvantummechanika sajátos jelenségeit — a szuperpozíciót, az összefonódást és az interferenciát — használja fel számítási erőforrásként. Ennek megértése nélkülözhetetlen a kriptográfiai következmények értékeléséhez.

1.2.1. A kvantumszámítás működési elvei

A klasszikus számítástechnika alapegysége a bit, amely kizárólag a 0 vagy az 1 értéket veheti fel. A kvantumszámítógép alapegysége a kvantumbit, röviden qubit, amelynek állapotát a kvantummechanika szabályai írják le. A qubit állapotát egy kétdimenziós komplex Hilbert-tér elemeként ábrázoljuk:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle,$$

ahol $\alpha, \beta \in \mathbb{C}$ és $|\alpha|^2 + |\beta|^2 = 1$. Ez azt jelenti, hogy mérés előtt a qubit egyszerre tartalmazza a $|0\rangle$ és a $|1\rangle$ állapotot, az ún. szuperpozíció elvének megfelelően. Mérés hatására a szuperpozíció összeomlik: $|\alpha|^2$ valószínűséggel kapjuk a 0-t, $|\beta|^2$ valószínűséggel az 1-et.

A szuperpozíció önmagában még nem tenné hatékonyá a kvantumszámítást. A másik kulcsfontosságú jelenség az összefonódás (entanglement): két qubit összefonódott állapotban olyan módon korrelált, hogy az egyik mérési eredménye azonnali hatással van a másikra, függetlenül a köztük lévő fizikai távolságtól. Az összefonódás következménye, hogy n qubit nem $2n$, hanem 2^n állapot egyidejű reprezentációjára képes. Egy 300 qubites rendszer több lehetséges állapotot reprezentál egyszerre, mint ahány atom van az ismert univerzumban.

A harmadik alapjelenség az interferencia. A kvantumalgoritmusokat úgy tervezik, hogy a helyes választ adó amplitúdók konstruktívan interferáljanak (erősítsék egymást), a helytelen válaszokat adó amplitúdók pedig destruktívan (gyengítsék egymást). Ez a mechanizmus teszi lehetővé, hogy a kvantumalgoritmusok exponenciálisan nagy keresési tereken belül hatékonyan találják meg a megoldást. A kvantumalgoritmus lényegében az amplitúdókat manipulálja kvantumkapukkal (unitér transzformációkkal), majd a végállapot mérésével nyeri ki a hasznos információt.

A kvantumszámítógépek fizikai megvalósítása számos különböző technológiai úton halad. A szupravezető qubitek (amelyeket az IBM, Google és mások fejlesztenek) mesterséges atomokat alkalmaznak szupravezető Josephson-kontaktuson alapuló áramkörökben, kriogén hőmérsékleten (≈ 15 mK). A csapdázott ionok (IonQ, Quantinuum) elektromágneses csapdákból tartott ionokat használják qubiteként, ami hosszabb koherenciaidőt és alacsonyabb hibaráttát eredményez. A fotonikus és topologikus kvantumszámítógépek szintén kutatás alatt állnak. 2019-ben a Google bejelentette a kvantumfölény elérését: a Sycamore processzor 53 qubittel egy mintavételi feladatot 200 másodperc alatt oldott meg, amelyhez a legjobb klasszikus szuperszámítógépek becsléseik szerint 10 000 évet igényeltek volna [1]. 2023-ra az IBM Eagle és Heron processzorai már 127, illetve 133 qubitet tartalmaztak, és a hibajavítással ellátott logikai qubitek is megjelentek.

A kvantumhiba-javítás a skálázhatóság egyik legnagyobb kihívása. Fizikai qubitek hibával terheltek: a koherenciaidő véges, a kapuhiba valószínűsége 10^{-3} – 10^{-4} nagyságrendű. A hibajavított logikai qubitek eléréséhez becslések szerint 1 000–10 000 fizikai qubit szükséges egyetlen logikai qubitenként, ami azt jelenti, hogy egy kriptográfiailag releváns,

néhány ezer logikai qubitet igénylő számítás elvégzéséhez millió fizikai qubit méretű gép szükséges. Ez ma még nem áll rendelkezésre, de a fejlesztési ütem gyors.

1.2.2. A kvantumszámítógépek előnyei és lehetőségei

A kvantumszámítástechnika potenciális alkalmazásai messze túlmutatnak a kriptográfiai fenyegetésen, és számos területen ígér a klasszikus számítástechnika fundamentális korlátainak áttörését.

A **molekulaszimulációk és kvantumkémia** területén a kvantumszámítógépek képesek molekulák elektronszerkezetét kvantummechanikai szinten szimulálni. Míg klasszikus számítógépen egy közepes méretű molekula pontos szimulációja exponenciálisan növekvő erőforrást igényel, addig egy kvantumszámítógép természetes hatókörébe esik a feladat. Ez közvetlen alkalmazást ígér a gyógyszerfejlesztésben (hatóanyag–fehérje kölcsönhatások), az anyagtudományban (új szupravezetők, akkumulátormateriák tervezése) és a katalízisvizsgálatban.

Optimalizálási feladatok esetén a kvantum heuristikák — mint a Quantum Approximate Optimization Algorithm (QAOA) és a Quantum Annealing — NP-nehéz problémák közelítő megoldására mutatnak ígéretes eredményeket. Logisztika, ellátásilánc-tervezés, pénzügyi portfólióoptimalizálás és menetrend-tervezés területén már kisebb méretű kísérletekben is demonstráltak kvantum előnyt.

A **pénzügyi modellezésben** a Monte Carlo-szimulációk kvantum gyorsítása négyzetgyök-csökkentést ígér a mintaszámban, ami kockázatbecslés és derivatív árazás területén jelent versenyelőnyt. Az **mesterséges intelligencia** területén kvantum-neuronhálózatokat és kvantum kernel-módszereket vizsgálnak, bár itt az előny kevésbé egyértelmű, mint az előző területeken.

Végül meg kell említeni a **kvantumkulcs-elosztást (QKD)**, amely a kvantummechanika törvényein alapuló biztonságos kulcscserét kínál: a kvantumállapotok lemásolhatatlansága (no-cloning tétel) és a mérés rendelkező hatása garantálja, hogy a lehallgatás kimutatható. A QKD azonban saját infrastruktúrát és speciális hardvert igényel, és nem közvetlenül helyettesíti a PQC-t.

1.2.3. Veszélyek a jelenlegi kriptográfiai rendszerekre

A kvantumszámítógépek kriptográfiai fenyegetése két alapvető algoritmuson keresztül materializálódik.

A **Shor-algoritmus** 1994-ben Peter Shor bebizonyította, hogy egy kvantumszámítógép polinomiális idő alatt képes az egészszámszorzás és a diszkrét logaritmus problémájának megoldására [13]. Az algoritmus kvantum-Fourier-transzformációt alkalmaz az adott szám prímtényezőinek meghatározásához. A faktorizálás időbonyolultsága $O((\log N)^3)$, szemben a legjobb ismert klasszikus szub-exponenciális algoritmussal, az általános számmező szitával (GNFS), amely körülbelül $O(\exp((\log N)^{1/3}))$ nagyságrendű. Ez közvetlenül aláássa az RSA, a Diffie–Hellman kulcscsere és az ECC biztonságát: az RSA-2048 feltöréséhez hozzávetőleg 4000 hibajavított logikai qubit szükséges — ez a szám a jelenlegi fejlesztési ütem alapján a 2030-as évekre lehet elérhető.

A **Grover-algoritmus** 1996-ban Lov Grover megmutatta, hogy egy N elemű rendezetlen adatbázisban a keresés kvantumszámítógépen $O(\sqrt{N})$ lépésben elvégezhető [6], szemben a klasszikus $O(N)$ lépéssel. Szimmetrikus kriptográfiánál ez a négyzetgyökgyorsulás azt jelenti, hogy egy 128 bites AES kulcs bruteforce-keresése olyan, mintha 64 bites kulcsra kellene keresni: a megoldás a kulcsméret megduplázása, azaz az AES-256 alkalmazása. A szimmetrikus kriptográfia tehát módosítással kvantum-ellenálló maradhat.

A legjelentősebb stratégiai veszélyt azonban a „**gyűjtsd most, fejtsd meg később**” (harvest-now, decrypt-later, HNDL) stratégia jelenti. Állami szintű szereplők és fejlett kiberbűnözői csoportok már most gyűjthetik a titkosított hálózati forgalmat, hogy azt egy jövőbeli kvantumszámítógéppel visszafejtsék. Azok az adatok, amelyeknek ma titkosítva kell maradniuk 10–20 éves időtávon (állami titkok, egészségügyi adatok, szellemi tulajdon, pénzügyi tranzakciók), már most célponttá válnak. Ez a fenyegetés sürgetővé teszi a PQC bevezetését még azelőtt, hogy a kvantumszámítógépek kriptográfiai szintre érnek.

A NIST 2016-ban meghirdetett versenye ennek a felismerésnek a közvetlen következménye volt. A hat éves értékelési folyamat eredményeként 2022-ben a NIST közzétette az első négy posztkvantum-kriptográfiai algoritmust: a CRYSTALS-Kyber-t (kulcskapcsolás), a CRYSTALS-Dilithium-ot és a FALCON-t (digitális aláírás), valamint a SPHINCS+-t (hash-alapú aláírás) [10]. 2024-ben ezek FIPS szabványként is megjelentek (FIPS 203, 204, 205).

1.3. A dolgok internete

Az IoT fogalma egy mélyen heterogén ökoszisztémát takar, amelynek határai az évek során folyamatosan tágulnak. Az egyszerű otthoni okoseszközöktől az ipari automatizálás vezérlőrendszerein át az orvosi implantátumokig terjed az eszközök köre, amelyek összekötik a fizikai és a digitális világot.

1.3.1. Az IoT kialakulása és fejlődése

Az „összes dolgot hálózatba kötni” ötlete valójában megelőzi az „Internet of Things” elnevezést. Mark Weiser, a Xerox PARC kutatója 1991-es *The Computer for the 21st Century* esszéjében lefektette az ubiquitous computing (mindenütt jelenlévő számítástechnika) koncepcióját: elképzelte, hogy számítástechnikai eszközök a mindennapi tárgyakba integrálódnak, és a felhasználó számára láthatatlanul végzik munkájukat [14]. Az 1990-es évek elején a Carnegie Mellon Egyetemen már létezett az első hálózatba kötött automatával bemutatott kísérlet, amely szenzorok segítségével tájékoztatta a felhasználókat az üdítőgép aktuális tartalmáról.

A „dolgok internete” elnevezést Kevin Ashton brit technológus alkotta meg 1999-ben, eredetileg az RFID (rádiófrekvenciás azonosítás) és az ellátáslánc-menedzsment összefüggésében. Az elgondolás lényege, hogy ha minden fizikai tárgyhoz egyedi azonosítót és hálózati kapcsolatot rendelünk, az objektumok maguk is adatforrásokká válnak, és a valós világ állapota automatikusan nyomon követhetővé válik [2]. Az RFID technológia lehetővé tette például a raktári készletezés automatizálását, amelyet a Procter & Gamble és az Auto-ID Center együttműködésével fejlesztett ki.

A 2000-es évek elejétől az ipari szenzorhálózatok (Wireless Sensor Networks, WSN) terjedtek el a gyártásban, a mezőgazdaságban és a katonai megfigyelésben. A fordulópontot az okostelefonok megjelenése (2007), az IPv6 szabvány széleskörű bevezetése, valamint az alacsony fogyasztású rádióprotokollok (Bluetooth LE, ZigBee, LoRaWAN, NB-IoT) és az olcsó ARM-alapú mikrovezérlők tömeges piaci megjelenése hozta el. 2010-től az IoT eszközök száma exponenciálisan növekedett: 2015-re elérte a 15 milliárdot, 2020-ra meghaladta a 30 milliárdot.

Az intelligens otthonok piacán a Nest termosztát (2011), az Amazon Echo (2014) és a Philips Hue izzósorozat paradigmaváltást hozott a fogyasztói elektronikában. Az ipari IoT a gyártásban és az energetikában a prediktív karbantartás, a folyamatoptimalizálás és a valós idejű monitorozás eszközévé vált. Az egészségügyi IoT területén a viselhető szenzoros eszközöktől a beültetett pacemaker-eken és szívritmus-figyelőkön át az infúziós pumpák

hálózati vezérléséig terjedt az alkalmazás. A McKinsey Global Institute becslése szerint az IoT gazdasági hatása 2030-ra 5,5–12,6 billió dollárt tesz ki évente [9].

1.3.2. IoT eszközök jellemzői és korlátai

Az IoT ökoszisztéma egyik legfontosabb sajátossága a rendkívüli heterogenitás és az erőforrás-korlátok jelenléte. Egy nagyvállalati szerver és egy ipari hőmérséklet-érzékelő között több nagyságrendnyi különbség lehet számítási teljesítményben, memóriában és energiaellátásban.

Tipikus korlátok az alacsony kategóriás IoT eszközöknél:

- **Processzor:** 8–32 bites mikrovezérlők (pl. ARM Cortex-M0/M4, Atmel AVR), órajel 8–120 MHz tartományban, hardveres lebegőpontos egység nélkül.
- **Memória:** 2–256 kB RAM, néhány kB – néhány MB flash tároló.
- **Energia:** elemről működő eszközöknél az aktív kriptográfiai számítás az egyik legnagyobb energiaigényű művelet, ezért fontos a kulcscsere-műveletek ritkítása.
- **Sávszélesség:** sok protokoll (LoRaWAN, NB-IoT) esetén a payload mérete néhány tíz–száz bájtá korlátozott, ami a PQC kulcs- és kapszulaméreteit érzékenyen érinti.
- **Üzemidő:** ipari és infrastrukturális eszközöket 10–20 éves élettartamra terveznek, ami azt jelenti, hogy a ma telepített eszközök a kvantumfenyegetés kritikus időszakában is aktívan üzemelni fognak.

Ez utóbbi szempont különösen fontos. Egy 2024-ben telepített, 15 éves élettartamra tervezett ipari szenzor 2039-ig működik. Ha a 2030-as évekre valóban megjelennek kriptográfiaileg releváns kvantumszámítógépek, ez az eszköz az aktív üzemeltetési ideje alatt válik sebezhető célponttá. Az ún. *cryptographic agility* — a kriptográfiai algoritmusok futás közbeni cserélhetőségének megőrzése a rendszerarchitektúrában — ezért az IoT tervezés egyik alapkövetelményévé vált.

1.3.3. IoT biztonsági kihívások

Az IoT biztonsági kérdései már jóval a kvantumfenyegetés előtt komoly figyelmet kaptak. A 2016-os Mirai botnet-támadás megmutatta, hogy hétköznapi IoT eszközök (IP-kamerák, DVR-ek) gyári jelszóval hagyott eszközei milyen pusztító erejű DDoS-támadáshoz nyújthatnak alapot: a botnet csúcsán másodpercenként 620 Gbps forgalmat generált, és az internetes infrastruktúra jelentős részét megbénította [8]. A 2021-es Oldsmar vízkezelőüzemi incidens, ahol egy támadó a klórszint módosításával próbálkozott, jól illusztrálta az IIoT rendszerek fizikai következményekkel járó sebezhetőségét.

A klasszikus kriptográfiai fenyegetéseken túl az IoT specifikus biztonsági kihívásai a következők:

Hitelesítési problémák. Erőforrás-korlátozott eszközökön a PKI alapú tanúsítványkezelés (TLS 1.3 futtatása, tanúsítványmegújítás, visszavonás-ellenőrzés) számottevő CPU-terhelést és memóriát igényel. Sok eszköz ezért gyári, nehezen cserélhető, megosztott titkokra támaszkodik, ami egypontos sebezhetőséget teremt.

Fizikai hozzáférés és oldalsáv-támadások. Az IoT eszközök fizikailag hozzáférhető helyen üzemelnek, fizikai biztonságuk korlátozott. A fogyasztásmérés-alapú (Differential Power Analysis, DPA) és elektromágneses sugárzáson alapuló (Electromagnetic Analysis, EMA) elemzések titkok kinyerésére alkalmasak, hacsak az implementáció nem állandó idejű (constant-time) és nem alkalmaz megfelelő zajellenintézkedéseket.

Firmware-frissítések és cryptographic agility. A hosszú üzemidő miatt az algoritmusváltásnak szoftverfrissítésen keresztül kell megvalósulnia. Ez előre megtervezett biztonságos bootloadereket, aláírt firmware-csomagokat és megbízható frissítési csatornákat feltételez. Ezek hiányában a deployed eszközök nem frissíthetők kvantum-ellenálló algoritmusokra.

Kulcskezelés és üzenetméret. Az IoT rendszerek skálája — sok milliós eszközpark — egyedi kulcsmenedzsment kihívásokat vet fel. A PQC implementáció során figyelemmel kell lenni arra, hogy a rács alapú és kód alapú algoritmusok nyilvános kulcsai és kapszulái lényegesen nagyobbak, mint az RSA vagy ECC megfelelőik, ami ütközhet az eszközök korlátozott tárhelyével és a sávszélesség-korlátokkal.

1.4. Posztkvantum-kriptográfia és IoT — a kapcsolat

A posztkvantum-kriptográfia azokat a kriptográfiai algoritmusokat foglalja magában, amelyek ellenállnak mind a klasszikus, mind a kvantumszámítógépek által nyújtott támadásoknak. Elnevezése némileg félrevezető: a „poszt-” előtag nem azt jelenti, hogy a kvantumszámítógépek már megjelentek, hanem azt, hogy az algoritmusok a kvantum utáni korszakra is biztonságosak.

A PQC főbb matematikai megközelítései:

- **Rács alapú kriptográfia:** az SVP és az LWE probléma nehézségén alapul. Ide tartoznak a CRYSTALS-Kyber (KEM) és CRYSTALS-Dilithium (aláírás), amelyeket a NIST 2022-ben szabványosított [10].
- **Kód alapú kriptográfia:** lineáris hibakijavító kódok dekódolásának NP-nehézségén alapul. A BIKE és a HQC ide sorolható; a McEliece-rendszer 1978 óta ismert, de kulcsméretei igen nagyok.
- **Hash alapú kriptográfia:** kiforrottsága és konzervatív biztonsági feltevései miatt kisebb aláírási rendszerek esetén vonzó (SPHINCS+).
- **Izogenián alapuló kriptográfia:** a SIKE algoritmust 2022-ben klasszikus támadással feltörték, ami e kategória korlátjaira hívja fel a figyelmet.

Az IoT kontextusában a PQC bevezetése különleges kihívásokat jelent. A CRYSTALS-Kyber-512 esetén a nyilvános kulcs 800 bájt, a kapszula 768 bájt, míg az RSA-2048 esetén ez 256 bájt — a háromszoros méretbeli növekedés mind a sávszélességet, mind a flash-tárhelyt igénybe veszi. Az elterjedt TLS és DTLS protokollok üzenetméreteinek módosítására is szükség van.

A számítási igény tekintetében kedvezőbb a kép: a CRYSTALS-Kyber és a BIKE ARM Cortex-M4 processzorral néhány tíz-száz milliszekundum alatt elvégezhetők [4, 3], ami a legtöbb IoT alkalmazásban elfogadható. A **hibrid megközelítés** — klasszikus és PQC kulcsere párhuzamos alkalmazása — a biztonsági migráció kockázatát is csökkenti: a kommunikáció biztonságos marad mindaddig, amíg legalább az egyik algoritmus ellenáll a fenyegetésnek. Ez ma az iparági ajánlás a PQC fokozatos bevezetésekor.

Az ipari és szabályozói nyomás is egyre erősebb. Az Európai Unió NIS2-irányelve és a Cyber Resilience Act kötelezővé teszik a beágyazott rendszerek gyártói számára a biztonsági frissíthetőség biztosítását és a kvantum-ellenálló kriptográfiára való tervezett átállást. Az IETF aktívan dolgozik a PQC-t integráló TLS és DTLS profilok kidolgozásán, különösen a CoAP és a 6LoWPAN kombinálásával működő IoT-profilok esetén [11].

Az összefonódó kihívások — a kvantumfenyegetés, az IoT korlátok és a szabályozói nyomás — együttesen sürgetik a PQC algoritmusok erőforrás-tudatos implementációinak

fejlesztését. E dolgozat éppen erre a hiányra ad választ: a CRYSTALS-Kyber és a BIKE algoritmusok IoT kontextusban való teljesítményének részletes mérésével és elemzésével.

1.5. A dolgozat céljai és felépítése

A dolgozat fő kutatási kérdése: *Hogyan alkalmazhatók a CRYSTALS-Kyber és BIKE posztkvantum-kriptográfiai algoritmusok erőforrás-korlátozott IoT eszközökön, és milyen teljesítménybeli különbségek mutatkoznak köztük valós mérési körülmények között?*

E kérdés megválaszolásához a következő célokat tűztük ki:

1. A CRYSTALS-Kyber és a BIKE algoritmusok elméleti hátterének és matematikai alapjainak részletes ismertetése, különös tekintettel az IoT-szempontról releváns implementációs kihívásokra.
2. Mérési környezet kialakítása, amely lehetővé teszi a két algoritmus valós erőforrás-korlátozott eszközön való teljesítmény-összehasonlítását.
3. A kulcsgenerálás, a kulcskapszulázás (encapsulation) és a visszafejtés (decapsulation) futási idejének, memóriaigényének és energiaszükségletének mérése és kiértékelése.
4. Következtetések levonása arról, melyik algoritmus alkalmasabb egyes IoT alkalmazási területekre, és milyen paraméterbeállítások optimálisak a biztonság és az erőforrásigény közötti kompromisszum szempontjából.

A dolgozat felépítése a következő. A **2. fejezet** részletesen tárgyalja a posztkvantum-kriptográfiai protokollok matematikai alapjait, a CRYSTALS-Kyber és a BIKE algoritmusok működési mechanizmusát és biztonsági tulajdonságait. A **3. fejezet** bemutatja a mérési környezetet: az alkalmazott hardvert, szoftverkörnyezetet és a wolfSSL kriptográfiai könyvtár PQC-implementációját. A **4. fejezet** leírja a mérési módszertant, a mért paramétereket és a mérési eljárást. Az **5. fejezet** tartalmazza a mérési eredményeket és azok részletes kiértékelését táblázatos és grafikus formában. A **6. fejezet** összefoglalja a következtetéseket, és javaslatokat fogalmaz meg a PQC IoT rendszerekben való alkalmazásának jövőbeli irányait illetően.

2. fejezet

Posztkvantum-kriptográfiai protokollok

Jelen fejezetben részletesen bemutatjuk a posztkvantum-kriptográfia két legígéretesebb protokollját, a CRYSTALS-Kyber és a BIKE (Bit Flipping Key Encapsulation) algoritmusokat. Ezek a protokollok a modern kriptográfiai kihívásokra adnak választ, előkészítve a rendszereinket a kvantumszámítógépek jövőbeli fenyegetésére. A fejezet célja, hogy megértsük a matematikai alapokat, a működési mechanizmusokat és az alkalmazási lehetőségeket, különös tekintettel az integrált rendszerek (IoT) biztonságára.

2.1. Posztkvantum-kriptográfia matematikai alapjai

A posztkvantum-kriptográfia azokra a kriptográfiai algoritmusokra utal, amelyek ellenállnak a kvantumszámítógépek támadásainak. A klasszikus kriptográfia biztonsága olyan matematikai problémák nehézségén alapul, amelyek klasszikus számítógépeken véges idő alatt nem oldhatók meg. Azonban a Shor-algoritmus felfedezése óta tudjuk, hogy a kvantumszámítógépek exponenciálisan felgyorsíthatják a faktorizációs és diszkrét logaritmus problémák megoldását, ami az RSA és az elliptikus görbe kriptográfia alapjait veszélyezteti.

A posztkvantum-kriptográfiai algoritmusok olyan matematikai struktúrára épülnek, amelyeknél még nem sikerült kvantum gyorsulást találni. Az egyik legfontosabb ilyen struktúra a rácselméleten alapuló titkosítás, amely a modul Learning With Errors (module-LWE) problémára épül.

2.1.1. A rácselmélet alapjai

A rács egy olyan diszkrét algebrai struktúra, amely a lineáris algebra és a számelmélet metszéspontjában helyezkedik el. Formálisan, egy n dimenziós rács a vektortérnek egy olyan részhalmaza, amely egy n vektorból álló bázis összes egész lineáris kombinációjaként állítható elő. Legyen $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n$ egy rács bázisa, akkor a rács pontjaira teljesül:

$$\Lambda = \left\{ \sum_{i=1}^n c_i \mathbf{b}_i \mid c_i \in \mathbb{Z} \right\}$$

A rácsok alapvető jellemzője, hogy számos nehéz matematikai probléma társul hozzájuk. Az egyik leghíresebb a legrövidebb vektor probléma (Shortest Vector Problem, SVP), amely azt kérdezi: adott egy rács, mekkora a nullvektortól különböző legrövidebb vektor? A másik központi probléma a legközelebbi vektor probléma (Closest Vector Problem, CVP), amely egy adott vektor rácshoz legközelebbi rácpontjának megkeresésére vonat-

kozik. Mindkét probléma NP-nehéznek bizonyult, és jelenleg nem ismeretes polinomiális idejű megoldása még kvantumszámítógépeken sem.

Az LWE probléma ezt az ötletet természetes módon a kriptográfiára adaptálja. Egy támadó abban az esetben sikeres, ha egy lineáris rendszer megoldását tudja helyreállítani úgy, hogy az egyenletek mellett hibás mérések is rendelkezésre állnak. Ez a probléma általánosítható moduláris polinomokra, amit module-LWE-nek hívunk, és ez képezi az alapját a Kyber protokollnak.

2.1.2. Az LWE probléma és annak moduláris változata

Az LWE (Learning With Errors) problémát Regev 2005-ben vezette be. Az alapfeladat a következőképpen fogalmazható meg: Tegyük fel, hogy van egy titok vektor $\mathbf{s} \in \mathbb{Z}_q^n$, és meg szeretnénk szerezni információt róla. Erre példa az $\langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q}$ egyenletek egy sorozata, ahol $\mathbf{a}$ egy véletlen vektor $\mathbb{Z}_q^n$ -ből, és e egy kis hiba $\mathbb{Z}_q$ -ből egy χ eloszlásból (tipikusan egy Gauss-eloszlás a tangens függvény kombinációjában).

Az LWE probléma azt kérdezi: polinomiális időben eldönthető-e, hogy egy $(A, \mathbf{b})$ pár, ahol $\mathbf{b} = A\mathbf{s} + \mathbf{e}$, vagy pedig a $\mathbf{b}$ csak egy véletlen vektor? Ez a döntési verzió, de létezik keresési verziója is, ahol az $\mathbf{s}$ vektort kell helyreállítani.

A Kyber protokoll a module-LWE problémát használja, amely az LWE-nek egy hatékonyabb változata. Ebben az esetben a vektorok helyett polinomokkal dolgozunk egy gyűrűben. A gyűrű általában $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, ahol n kettő hatványa és q egy nagyobb prímszám. Az $X^n + 1$ polinomot választjuk, mert a NTT (Number Theoretic Transform) algoritmussal hatékonyan lehet vele számolni.

A module-LWE probléma a következőképpen írható fel: Adott az $\mathbf{A} \in R_q^{k \times \ell}$ egy véletlen mátrix polinomokkal, az $\mathbf{s} \in R_q^\ell$ egy titok vektor, és az $\mathbf{e} \in R_q^k$ egy kis hibavektor, meg kell-e különböztetni az $(\mathbf{A}, \mathbf{b})$ párot, ahol $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, egy $(A, \mathbf{b}')$ párotól, ahol $\mathbf{b}'$ véletlen? Az LWE és module-LWE problémákat széles körben tanulmányozták, és jelenleg semmilyen klasszikus vagy kvantum algoritmus nem ismert, amely polinomiális időben megoldaná őket.

2.2. CRYSTALS-Kyber protokoll

A CRYSTALS-Kyber (Crystal Lattice Signatures for Encryption and Signatures - Kyber) a posztkvantum-kriptográfia egyik legígéretesebb megoldása. A protokoll 2022-ben standardizálva lett a NIST által, és széles körben használható a gyakorlati alkalmazásokban. A Kyber egy kulcsbecsapódás mechanizmus (Key Encapsulation Mechanism, KEM), amely lehetővé teszi, hogy két fél közösen megosztott titkot hozzon létre, amely később szimmetrikus titkosításhoz használható.

2.2.1. Paraméterek és biztonsági szintek

A Kyber protokoll különböző biztonsági szinteket kínál, amely az alkalmazás igényeihez igazítható. Az alap paraméterek:

- **Kyber512:** 128 bites klasszikus biztonsági szint, amely az AES-128 szintű biztonságna felel meg
- **Kyber768:** 192 bites klasszikus biztonsági szint, amely az AES-192 szintű biztonságna felel meg
- **Kyber1024:** 256 bites klasszikus biztonsági szint, amely az AES-256 szintű biztonságna felel meg

Az egyes szintekhez különböző paraméterek tartoznak. A Kyber512 a következő paramétereket használja: $n = 256$, $q = 3329$, $k = 2$ (ahol k az $\mathbf{A}$ mátrix dimenziója), $\eta_1 = 3$, $\eta_2 = 2$. A Kyber768 paraméterek: $n = 256$, $q = 3329$, $k = 3$, $\eta_1 = 2$, $\eta_2 = 2$. A Kyber1024 paraméterek: $n = 256$, $q = 3329$, $k = 4$, $\eta_1 = 2$, $\eta_2 = 2$.

Az η_1 és η_2 paraméterek határozzák meg a hibavektorok egy közép-re igazított binomiális eloszlásának szórásának szintjét. Ezek az értékek kritikusak a biztonsági szint garantálásához, mert az túl kicsi hibák a protokollt törhetővé teszik, míg a túl nagy hibák viszont a decapsulation megbízhatóságát csökkentik.

2.2.2. Kulcscsere és kulcsencapsulation

A Kyber protokoll három fő szubrutinból áll: KeyGen, Encaps, és Decaps.

2.2.2.1. Kulcsgenerálás (KeyGen)

A kulcsgenerálás során az egyik fél (Alice) elkészíti a nyilvános és privát kulcsot. Az algoritmus a következő lépésekből áll:

1. Sorsoljunk le egy véletlenszerű értéket $d \in \{0, 1\}^{32}$ (ezt a seed-et)
2. A d seed-ből váltsan ki az ρ értéket a SHA3-512 hash függvényvel
3. Az ρ értékből generáljunk egy véletlen mátrixot $\mathbf{A} \in R_q^{k \times k}$ az XOF (extendable output function) beépített generátorral
4. Generáljunk egy véletlenszerű secretet $\mathbf{s}, \mathbf{e} \in R_q^k$ binomiális eloszlásból az η_1 paraméterrel
5. Számítsuk ki a $\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \in R_q^k$ értéket
6. A nyilvános kulcs: $pk = (\mathbf{t}, \rho)$
7. A privát kulcs: $sk = (\mathbf{s}, \mathbf{e})$, valamint további technikai adatok

Az algoritmus során a $\mathbf{A}$ mátrix számítása a kiterjesztett kimeneti függvény segítségével történik, amely biztosítja, hogy az mátrix pszeudó-véletlen marad, de reprodukálható.

2.2.2.2. Üzenet encapsulation (Encaps)

A Kyber encapsulation során egy kulcscsatorna készül. Bob, aki Alice nyilvános kulcsát ismeri, a következő lépéseket hajtja végre:

1. Sorsoljunk le egy véletlenszerű üzenet $m \in \{0, 1\}^{32-t}$
2. A m üzenetből váltsan ki G hash függvényvel egy $(r, d') \in \{0, 1\}^{32} \times \{0, 1\}^{32}$ párt
3. Generáljunk egy véletlenszerű vektort $\mathbf{y} \in R_q^k$ binomiális eloszlásból az η_1 paraméterrel
4. Számítsuk ki a $\mathbf{u} = \mathbf{A}^T \cdot \mathbf{y} + \mathbf{e}' \in R_q^k$ értéket, ahol $\mathbf{e}' \in R_q^k$ binomiális eloszlásból az η_2 paraméterrel
5. Számítsuk ki a $\mu = \lfloor \frac{q}{2} \rfloor \cdot m_i$ értéket minden i -re, és a $v = \mathbf{t}^T \cdot \mathbf{y} + e'' + \mu$ értéket, ahol e'' egy skalár hibavektor
6. A kapszula (ciphertext): $c = (\mathbf{u}, v)$

7. A megosztott titok (shared secret): $K = H(m)$, ahol H a SHA3-256 függvény

Az encapsulation folyamata garantálja, hogy az üzenet m titkos marad, mivel csak Alice, aki ismeri az s vektort, tudja helyreállítani az eredeti üzenetet.

2.2.2.3. Üzenet decapsulation (Decaps)

Az Alice által végrehajtott decapsulation az encapsulation inverze. A nyilvános kapszula és a privát kulcs alapján:

1. Számítsuk ki a $\mathbf{u}'$ és v' értékeket az s vektort használva: $\mathbf{u}' = \mathbf{s}^T \cdot \mathbf{u} \in R_q^k \pmod{q}$
2. Számítsuk ki a hibahatáron belüli üzenet biteket: $m'_i = 1$ ha $|v - \mathbf{u}' - \lfloor \frac{q}{2} \rfloor|_q > \lfloor \frac{q}{4} \rfloor$, egyébként $m'_i = 0$
3. Az helyreállított üzenet: $m' = \text{recover}(\mathbf{u}', v)$
4. A megosztott titok: $K = H(m')$

Az LWE probléma nehézsége biztosítja, hogy az s vektor nélkül nem lehet helyreállítani az üzenetet, még akkor sem, ha az encapsulation alatt zajló számítások részleges információi rendelkezésre állnak.

2.2.3. Biztonsági tulajdonságok és hardver gyorsulás

A Kyber biztonsága az module-LWE probléma feltételezett nehézségén alapul. A NIST szabványosítási folyamatban az összes benyújtott posztkvantum algoritmussal szembeni támadások részletes elemzésére irányuló kutatások azt mutatják, hogy a Kyber protokoll az elvárt biztonsági szinteket nyújtja, ha a paraméterek helyesen kerülnek megválasztásra. A Kyber különösen sérülékeny az oldalsávós támadásokra (side-channel attacks), így az implementációnak óvatosságnak kell lennie az időzítési információk elkerüléséhez.

A Kyber nagy előnye az NTT algoritmusnak köszönhető, amely a gyűrűn végzett szorzásokat lineáris időre csökkenti. Az NTT a Fast Fourier Transform (FFT) módosított verziója, amely egész számok fölött működik a q prímszám feldolgozásával. A modern processzorok (mint az Intel vagy ARM) támogatják a vektoros utasításokat (SIMD - Single Instruction Multiple Data), amelyekkel a Kyber hatékonyan felgyorsítható. Az optimalizált implementációk másodpercenként több száz kulcscserét tudnak végezni standard laptop processzorokon.

2.3. BIKE (Bit Flipping Key Encapsulation) protokoll

A BIKE egy alternatív posztkvantum-kriptográfiai protokoll, amely az error-correcting codes elméletén alapul. Szemben a Kyber rács-alapú megközelítésével, a BIKE kód-alapú kriptográfiát alkalmaz, amely a McEliece rendszer eszméire épül. A kód-alapú titkosítás hosszú történelmi gyökerekkel rendelkezik, és erős ellenálló képességgel bír a kvantum-támadásokkal szemben.

2.3.1. Kód-alapú titkosítás alapelvei

A kód-alapú kriptográfia fundamentális ötlete, hogy az error-correcting kódok dekódolása általános esetben NP-nehéz probléma. Az egyik legfontosabb result, hogy szinte minden véletlen kód dekódolása NP-teljes. Ez azt jelenti, hogy nem ismeretes polinomiális idejű algoritmus az általános eset dekódolására, még kvantum-támadások alatt sem.

A McEliece sistem, amely 1978-ban lett bevezetett, ezt az ötletet használta. A titkos kulcs az egy könnyen dekódolható kód (például Goppa-kód), a nyilvános kulcs pedig a kód generátor mátrixának egy permutált és zavart verziója. Ennek a rendszernek az a nagy előnye, hogy az encapsulation rendkívül gyors (csak mátrix-vektor szorzás és hibahozzáadás), azonban a kulcsméretetek nagyok (több tízezer bit).

A BIKE ezt a koncepciót modernizálja az MDPC (Moderate-Density Parity-Check) kódok felhasználásával, amelyek kompaktabb kulcsokat lehetővé tesznek az error-correcting kód-alapú titkosítás megtartása mellett.

2.3.2. MDPC kódok és az BIKE paraméterek

Az MDPC kódok olyan lineáris kódok, amelyekben a paritás-mátrix ritka (sparse), vagyis csupán néhány nem-nulla elemmel rendelkezik minden sorban. Formálisan, egy n hosszú MDPC kód az H paritás-mátrixszal definiálható, ahol az H egy $m \times n$ mátrix, és az egyes soroknak csupán d_c nem-nulla elemei vannak (constant weight).

Adott egy hibavektor $\mathbf{e} \in \mathbb{F}_2^n$ (ahol az elemek 0 vagy 1), az $\mathbf{e}$ a kódhoz tartozik, ha $H \cdot \mathbf{e}^T = \mathbf{0}^T \pmod{2}$. Ha az $\mathbf{e}$ -nak néhány nem-nulla bitje van (például t darab), akkor az $H \cdot \mathbf{e}^T$ nem lesz zérus, hanem az úgynevezett szindróma (syndrome).

Az MDPC dekódolási probléma azt kérdezi: adott egy szindróma $\mathbf{s} = H \cdot \mathbf{e}^T$, és az H paritás-mátrix, helyreállítható-e az eredeti hibavektor $\mathbf{e}$ a rendszeres dekódolási algoritmusokkal? Ha a hibaváltság és a kód paraméterei helyesen kerülnek választásra, a dekódolás szinte garantált, de egy támadónak (aki az H teljes szerkezetét nem ismeri) gyakorlatilag lehetetlen.

A BIKE protokoll különböző szinteket kínál:

- **BIKE-L1**: 128 bites klasszikus biztonsági szint, $n = 12323$, $d = 71$, $t = 134$
- **BIKE-L3**: 192 bites klasszikus biztonsági szint, $n = 24659$, $d = 71$, $t = 199$
- **BIKE-L5**: 256 bites klasszikus biztonsági szint, $n = 40973$, $d = 71$, $t = 264$

Itt az n a kód hossza, az d az MDPC kód soros súlya (row weight), és az t az a maximális hiba-számlálás, amit a dekódoló még helyesen kezel. Az n és t értékek közvetlenül kapcsolódnak a biztonsági szinthez.

2.3.3. BIKE kulcsgenerálás és encapsulation

A BIKE protokoll szintén három fő algoritmusból áll: KeyGen, Encaps, és Decaps.

2.3.3.1. Kulcsgenerálás (KeyGen)

A BIKE kulcsgenerálása az MDPC kódok szerkezetéből indul:

1. Generáljunk egy véletlenszerű ritka mátrixot $H = [H_0, H_1]$, ahol az H_0 és H_1 az $m \times n$ alakú, és az egyes soroknak pontosan d nem-nulla elemük van (constant weight)
2. Az H_0 sorainak az inverze (ha létezik) az L mátrix
3. Az H -ből számítsuk ki az effektív paritás-mátrixot: $H' = L \cdot H = [I, H']$ (identity és egy H' mátrix)
4. A nyilvános kulcs: $pk = H'$ (vagy annak egy tömörített reprezentációja)
5. A privát kulcs: $sk = H$ (az eredeti paritás-mátrix és az L inverz mátrix információi)

2.3.3.2. Üzenet encapsulation (Encaps)

A Kyber-hez hasonlóan, az encapsulation egy megosztott titkot hoz létre:

1. Sorsoljunk le egy véletlenszerű üzenet $m \in \{0, 1\}^n$ bits, de csak az első t bit legyen nem-nulla
2. A szindrómát számítsuk ki: $\mathbf{s} = H' \cdot \mathbf{m}^T \pmod{2}$
3. A kapszula: $c = \mathbf{s}$ (vagy egy tömörített verzió)
4. A megosztott titok: $K = H(\mathbf{m})$

2.3.3.3. Üzenet decapsulation (Decaps) és bit-flipping dekódolás

A decapsulation során az Alice ismeri az eredeti H mátrixot, így helyreállíthatja az üzenetet:

1. A szindrómából indulva ($\mathbf{s}$), a dekódoló algoritmus helyreállítja az eredeti hibavektort $\mathbf{e}'$
2. A helyreállított üzenet: $\mathbf{m}' = \mathbf{e}'$
3. A megosztott titok: $K = H(\mathbf{m}')$

A bit-flipping algoritmus az egyik legegyszerűbb és leggyakrabban használt MDPC dekódolási módszer. Az algoritmus iteratívan működik:

1. Inicializálás: az aktuális hibavektor $\mathbf{e}_0 = \mathbf{0}$
2. Iteráció $i = 1, 2, \dots, \text{max_iter}$:
 - (a) Számítsuk ki az aktuális szindrómát: $\mathbf{s}_i = \mathbf{s} + H \cdot \mathbf{e}_i^T \pmod{2}$
 - (b) Az j pozícióban lévő hiba-score: $\text{score}_j =$ az j -ik bit által befolyásolt szindróma bitek száma
 - (c) A legmagasabb score-ú pozícióban fordítsuk az $\mathbf{e}_{i+1}$ bitet: $e_{i+1,j} = e_{i,j} \oplus 1$ ha $\text{score}_j \geq \text{threshold}$
 - (d) Ha az $\mathbf{s}_i = \mathbf{0}$, akkor sikeresen dekódoltunk
3. Ha az iterációs limit elérésre kerül, és még nem színgátunk, az algoritmus megghiúsul vagy egy feltételes választási módszert alkalmaz

A bit-flipping algoritmus hatékonysága az $\mathbf{m}$ hibavektor súlyától és a küszöb (threshold) választásától függ. A megfelelő paraméterek mellett az algoritmus szinte garantáltan helyreállítja az t nagyságrendű hibavektort. Az algoritmus nagy előnye az $O(n^2t)$ időbonyolultsága az H mátrix elég ritka volta miatt, amely lehetővé teszi az ilyen nagyméretű kódok praktikus alkalmazását.

2.3.4. Biztonsági tulajdonságok

A BIKE biztonsága az MDPC dekódolási probléma és az általános szindróma dekódolás NP-nehézségén alapul. Az ismeretes támadások közül az egyik legeredményesebb az úgynevezett *decoding failure attack*, amely azt próbálja kihasználni, hogy az MDPC dekódoló néha megghiúsulhat. Az újabban bevezetett kutatások azt mutatják, hogy a BIKE paraméterek a szükséges biztonsági szinteket nyújtják, ha az encapsulation transzformáció

helyesen (például az Fujisaki-Okamoto transzformációval) kerül meghívásra az IND-CCA biztonsági garantálásához.

A BIKE egy másik előnye az implementációs egyszerűsége. Az encapsulation és az initial decoding csupán bitwise XOR és összeadást igényel, ami könnyen felgyorsítható szoftveres és hardveres eszközökkel. A Kyber-hez képest a BIKE nagyobb kulcsméreteket igényel, azonban az encapsulation részben gyorsabb lehet egyszerűbb eszközökön.

2.4. Oldalsávós támadások és biztonsági implementáció

A posztkvantum-kriptográfiai algoritmusok biztonságát nemcsak az elméleti matematikai alapjai, hanem a gyakorlati implementáció is meghatározza. Az oldalsávós támadások (side-channel attacks) olyan támadási módszerek, amelyek a fizikai implementáció melléktermékeit, mint az időzítési információ, energiafogyasztás vagy elektromágneses sugárzás kihasználják.

2.4.1. Időzítési támadások (Timing Attacks)

Az időzítési támadások azt használják ki, hogy az algoritmus futási ideje függhet a feldolgozott adatoktól. Például, ha az LWE dekódoló algoritmus gyorsabban fejeződik be bizonyos hibaértékek mellett, akkor egy támadó a futási időmérésből információkat szerezhet az eredeti titkos kulcsról.

A Kyber implementáció során különös figyelmet kell fordítani arra, hogy az összes operáció azonos időben fusson le, függetlenül az input értékeitől. Ez az úgynevezett konstant-time implementáció. Az NTT algoritmus például nem konstant-time természet szerint, ezért az implementáció során dummy operációkat kell beszúrni az időzítés kiegyenlítéshez.

A BIKE esetén a bit-flipping dekódoló algoritmus szintén sérülékeny az időzítési támadásokra, mivel az iteráció száma az aktuális hibaminta sikerességétől függ. Az ajánlott megoldás a dekódolási iterációk számának rögzítése, még akkor is, ha az algoritmus korábban sikeresen helyreállított egy érvényes hibavektort.

2.4.2. Energiafogyasztási támadások (Power Analysis)

Az energiafogyasztási támadások az eszköz elektromos energiafogyasztásának megfigyelésén alapulnak. A különféle operációknak (összeadás, szorzás, memória-hozzáférés) különböző energiafogyasztása van, és ezek a különbségek felhasználhatók az algoritmus belső állapotaival kapcsolatos információkra.

A CPA (Correlation Power Analysis) a legerősebb ismert energiafogyasztási támadás, amely statisztikai módszereket használ a megfigyelett energiajelek és az ismert algoritmus-kimenet közötti korrelációk feltérképezésére. A Kyber és BIKE implementációknak maszkozási technikákat kell alkalmazniuk, ahol az összes kritikus adat logikai maszkokkal kombinálódik, hogy az energiajelek ne legyenek korrelálva az feldolgozandó érzékeny információkkal.

2.4.3. Hibabeviteli támadások (Fault Injection)

A hibabeviteli támadások során az támadó szándékosan hibákat vezet be az eszköz működésébe, például elektromágneses impulzus vagy lézer segítségével, és megfigyelik, hogy az algoritmus hogyan reagál. A hibázó számítások eredménye információt árulhat el az algoritmus titkos paramétereiről.

Az Kyber és BIKE protokolloknak ellenállónak kell lenniük az ilyen támadásokkal szemben. Az egyik megközelítés a redundanciás számítások: az algoritmus több alkalom-

mal fut, és az eredmények egyezéseit ellenőrzik. Ha egy hiba detektálódik, az algoritmus meghiúsul vagy kijelent egy hibát, helyett hogy helytelen eredményt adna.

2.5. IoT alkalmazások és integráció

Az Internet of Things (IoT) rendszerek gigantikus mértékben terjednek, és a biztonsági megközelítés kritikus fontosságú. Az IoT eszközök között vannak erőforrás-korlátozott mikrokontrollerek (például ARM Cortex-M4), melyeknek korlátozott memória (néhány kilobájt RAM), korlátozott feldolgozási teljesítmény és alacsony energiabáziú működés jellemző.

2.5.1. Kyber IoT implementáció

A Kyber protokoll viszonylag kis kulcsméretei és jó párhuzamosíthatósága miatt alkalmas az IoT alkalmazásokra. Az ARM Cortex-M4 processzor támogatja a SIMD szerű utasításokat, amelyekkel a Kyber szorzásokat jellegzetes módon felgyorsítható. Az OpenQuantumSafe és a liboqs-python nyílt forráskódú implementációi már biztosítanak optimalizált kódokat az ARM platformokra.

A Kyber512 paraméterek egy alap IoT eszközre teljesíthetők körülbelül 10-20 milliszekundumos encapsulation és decapsulation idővel, ami az legtöbb alkalmazás számára elegendő. Az energiafogyasztás egy "embedded" kontextusban körülbelül 10-50 millijoule egy teljes kulcscsere művelet során, ami elfogadható az legtöbb IoT alkalmazásban.

2.5.2. BIKE IoT implementáció

A BIKE protokoll nagyobb kulcsméreti és iteratív dekódolási folyamata miatti kihívásokat jelentenek az erőforrás-korlátozott IoT eszközökön. Azonban az encapsulation művelet (amely a küldő oldalon történik) viszonylag gyors és energiahatékony, mivel csupán bit-szintű operációkra támaszkodik. Az erőforrás-korlátozott szándékolt decapsulation (a fogadó oldalon) azonban több időt és energiát igényelhet.

Egy Cortex-M4-en a BIKE-L1 paraméterekkel végzett decapsulation tipikusan 100-200 milliszekundum időt vesz igénybe, ami bizonyos alkalmazásokra még elfogadható, de az encapsulation-hoz képest lassabb.

2.5.3. Hibrid megközelítés az IoT-ben

Az ajánlott gyakorlat az IoT rendszerekben egy hibrid kulcscseremechanizmus, amely kombinálja az elliptikus görbe Diffie-Hellman (ECDH) kulcscserét egy posztkvantum-kriptográfiai protokollal (ML-KEM/Kyber vagy ML-KEM-BIKE). Ez azt jelenti, hogy az IoT eszköz két független kulcscsatornát létesít:

1. **Klasszikus csatorna:** ECDH kulcscsere, amely ma biztonságos klasszikus támadások ellen
2. **Posztkvantum csatorna:** ML-KEM (Kyber) kulcsencapsulation, amely biztonságos a jövőbeli kvantum-támadások ellen

A végső szimmetrikus kulcs a két csatorna kimenetének kombinációja:

$$K_{\text{final}} = H(K_{\text{ECDH}} \| K_{\text{ML-KEM}})$$

Ez a megközelítés biztosítja, hogy még akkor is, ha az egyik protokoll valahogy kompromittálódna, a rendszer biztonsága az másik algoritmus ereje által védett marad. Ezenkívül biztosítja, hogy az a mai klasszikus támadásokkal szemben megmarad az ECDH biztonság, valamint a jövőbeli kvantum-veszélyek előtt a Kyber védelmez.

2.5.4. Memória- és tárolási követelmények

Az IoT eszközök korlátozottak a tárcsapásméreteiben és memóriájukban. A Kyber512 nyilvános kulcsának mérete körülbelül 800 bájt, a privát kulcsé pedig 1632 bájt. Ez az legtöbb IoT eszközre közvetlenül Flash memóriában tárolható.

A BIKE-L1 paraméterek nagyobb kulcsokat igényelnek: körülbelül 1541 bájt nyilvános kulcs és 3104 bájt privát kulcs. Az erőforrás-korlátozott eszközökön ez szűkös lehet, és tömörítési technikákat (például az MDPC mátrix kompakt reprezentációja) lehet alkalmazni a méret csökkentésére.

Az encapsulation során létrehozott kapszula mérete szintén fontos az IoT kommunikációban. A Kyber512 kapszulájának mérete körülbelül 768 bájt, míg a BIKE-L1 kapszulájának mérete körülbelül 1541 bájt. Mivel az IoT hálózatokban az adatok átvitele költséges (energiafogyasztás és sávszélesség szempontjából), a kisebb kapszula-méretnek előnyösebbek.

2.6. Kyber és BIKE összehasonlítása

A két protokoll közötti választás több tényezőtől függ: a biztonsági követelményektől, az alkalmazás teljesítmény-kritikussága, és az elérhető hardver-támogatástól.

2.6.1. Biztonsági megfontolások

Mind a Kyber, mint a BIKE széles körben tanulmányozottak és alapvető matematikai problémák NP-nehézségén alapulnak. Az LWE probléma és az MDPC dekódolás egyaránt nem ismert kvantum-gyorsulást. A NIST standardizálási folyamatban mindkettő végig jól teljesített, de végül a Kyber lett az ajánlott KEM standard az ML-KEM megnevezéssel.

2.6.2. Teljesítmény- és méret-jellemzők

A Kyber általában kisebb kulcsokat igényel az BIKE-hez képest. Például Kyber768 esetén a nyilvános kulcs mérete körülbelül 1184 bájt, míg BIKE-L3 esetén 1541 bájt. Az encapsulation sebesség hasonló, azonban a decapsulation során a Kyber nagy szorzások végzésénél a BIKE inkább bit-szintű operációkra támaszkodik.

2.6.3. Gyakorlati alkalmazások

Az IoT rendszerekben a Kyber előnyösebb az kisebb kulcsméretei és jó párhuzamosíthatósági lehetőségei miatt. Az olyan erőforrás-korlátozott eszközökön, mint az ARM Cortex-M mikrokontrollerek, a Kyber szoftver-szintű optimalizálása már megtörtént, és elérhető nyílt forráskódú implementáció.

A BIKE-nek nagyobb memóriefoglalata van, azonban az olyan speciális alkalmazásokban, ahol az encapsulation sebesség kritikus (például egy nagyon magas kódolási rátájú csatorna), szintén értékes lehet.

2.6.4. Hibrid megközelítés

A gyakorlatban az ajánlott megközelítés egy hibrid rendszer, amely klasszikus és poszt-kvantum kriptográfiai algoritmusokat kombinál. Például egy ECDH (Elliptic Curve Diffie-Hellman) kulcscsere mellett egy ML-KEM (Kyber) kulcscsatornát végeznek, és a végső szimmetrikus kulcs mindkettő kombinációja. Ez biztosítja, hogy még ha az egyik algoritmus kompromittálódna is (akár egy nem várt felfedezés, akár egy kvantum támadás miatt), a rendszer biztonsága megmarad az másik algoritmus ereje által.

3. fejezet

Mérési környezet

Jelen fejezet a kísérleti mérések elvégzéséhez kialakított hardver- és szoftverkönyezetet mutatja be. A fejezet célja, hogy az eredmények reprodukálhatóságához szükséges összes technikai részletet rögzítse: az alkalmazott eszközök specifikációját, a szoftverkomponensek verzióit és a hálózati topológiát.

3.1. Hardverkönyezet

3.1.1. Tesztplatform specifikációja

3.1.2. Hálózati topológia

3.1.3. Energiaellátás és mérőeszközök

3.2. Szoftverkönyezet

3.2.1. Operációs rendszer és alapkönyvtárak

3.2.2. Kriptográfiai könyvtárak

3.2.3. MQTT infrastruktúra

3.2.4. Konténerizáció és reprodukálhatóság

3.3. A mérési környezet összefoglalása

4. fejezet

Mérés metodotógia

4.1. Mosquitto alapú MQTT laboratóriumi környezet kialakítása posztkvantum kriptográfiai támogatással

4.1.1. Bevezetés

Az Internet of Things (IoT) rendszerek terjedésével egyre nagyobb jelentősége van a könnyűsúlyú és megbízható kommunikációs protolloknak. Az egyik legelterjedtebb ilyen protokoll az MQTT (Message Queuing Telemetry Transport), amelyet kifejezetten kis erőforrásigényű eszközök és instabil hálózati kapcsolatok számára terveztek. Az MQTT protokoll publish-subscribe kommunikációs modellt használ, amely lehetővé teszi, hogy az eszközök közvetlen kapcsolat nélkül, egy központi broker segítségével kommunikáljanak egymással.

Az MQTT széles körben alkalmazott technológia okosotthon rendszerekben, ipari automatizálásban, szenzorhálózatokban és edge computing környezetekben. Mivel az ilyen rendszerek gyakran érzékeny adatokat továbbítanak, kiemelt szerepe van a biztonságos kommunikációnak. Jelenleg a legtöbb TLS-alapú kommunikáció hagyományos nyilvános kulcsú algoritmusokra, például RSA-ra vagy elliptikus görbéken alapuló kriptográfiára épül. Ezeket a megoldásokat azonban a jövőben veszélyeztethetik a kvantumszámítógépek.

A posztkvantum kriptográfia (Post-Quantum Cryptography – PQC) olyan algoritmusok fejlesztésével foglalkozik, amelyek ellenállnak a kvantumszámítógépek támadásainak is. Az Amerikai Szabványügyi és Technológiai Intézet (NIST) több évig tartó standardizációs folyamat során választotta ki azokat az algoritmusokat, amelyek várhatóan a jövő kvantumbiztos rendszereinek alapját képezik majd. Ezek közé tartozik például az ML-KEM (korábban Kyber) és az ML-DSA (korábban Dilithium).

A laboratóriumi környezet célja egy olyan MQTT infrastruktúra kialakítása volt, amelyben a kommunikáció TLS kapcsolaton keresztül, posztkvantum algoritmusok támogatásával valósul meg. A labor lehetővé teszi a PQC algoritmusok gyakorlati tesztelését, kompatibilitási vizsgálatát, valamint későbbi teljesítménymérések végrehajtását.

4.1.2. A labor célja és architektúrája

A labor célja egy reprodukálható, izolált MQTT tesztkörnyezet létrehozása volt, amely támogatja a posztkvantum kriptográfiai algoritmusok használatát TLS kapcsolatokban.

A kialakított környezet fő komponensei:

- Ubuntu Linux alapú rendszer
- Docker alapú izoláció

- Egyedi fordítású OpenSSL 3.3
- Open Quantum Safe (OQS) projekt komponensei
- Eclipse Mosquitto MQTT broker
- TLS kommunikáció PQC algoritmusokkal

A rendszer logikai felépítése a következő:

1. MQTT kliens
2. TLS kapcsolat
3. OpenSSL könyvtár
4. OQS Provider
5. Mosquitto broker

A Docker alapú megközelítés biztosítja, hogy a labor könnyen újraépíthető és más rendszereken is reprodukálható legyen. Ez különösen fontos kutatási környezetben, ahol az eredmények reprodukálhatósága alapvető követelmény.

4.1.3. Operációs rendszer és alapsomagok telepítése

A labor Ubuntu 24.04 alapú Linux környezetben készült. Az Ubuntu választása több szempontból is előnyös:

- széles körű dokumentációval rendelkezik,
- stabil fejlesztői környezetet biztosít,
- jól támogatja a Docker és OpenSSL build folyamatokat,
- kompatibilis az Open Quantum Safe projekt komponenseivel.

A telepítést követően első lépésként a rendszer frissítése szükséges. Ez biztosítja, hogy minden csomag naprakész legyen, valamint csökkenti a kompatibilitási problémák kockázatát.

```
sudo apt update
sudo apt upgrade -y
```

A build folyamatokhoz és a fejlesztői környezet kialakításához több csomag telepítése szükséges.

```
sudo apt install -y \
build-essential \
cmake \
git \
ninja-build \
perl \
python3 \
python3-pip \
libtool \
autoconf \
automake
```

A `build-essential` csomag biztosítja a GCC fordítót és az alapvető fejlesztői eszközöket. A `cmake` modern build rendszert biztosít, míg a `ninja-build` gyorsabb fordítást tesz lehetővé. Az OpenSSL build folyamata Perl scriptet használ, ezért a `perl` csomag szintén szükséges.

4.1.4. Docker környezet kialakítása

A Docker egy konténerizációs platform, amely lehetővé teszi izolált futtatási környezetek létrehozását. A laborban a Docker használata több okból is indokolt:

- elkülöníti a PQC OpenSSL buildet a hoszt rendszertől,
- egyszerűsíti a függőségek kezelését,
- biztosítja a reprodukálható környezetet,
- lehetővé teszi a labor gyors újraépítését.

A Docker telepítése:

```
sudo apt install -y docker.io docker-compose
```

A Docker szolgáltatás automatikus indítása:

```
sudo systemctl enable docker
sudo systemctl start docker
```

A felhasználó hozzáadása a Docker csoporthoz:

```
sudo usermod -aG docker $USER
```

Ezután új bejelentkezés szükséges, hogy a jogosultságok érvénybe lépjenek.

4.1.5. Az Open Quantum Safe projekt komponensei

A labor egyik legfontosabb eleme az Open Quantum Safe projekt használata. Az OQS projekt célja a posztkvantum algoritmusok integrálása valós rendszerekbe és protollokba.

A laborban két fő komponens került felhasználásra:

- `liboqs`
- `oqs-provider`

A `liboqs` könyvtár implementálja a különböző PQC algoritmusokat. Ide tartoznak például az ML-KEM, HQC, BIKE és más NIST standardizáció alatt álló algoritmusok.

Az `oqs-provider` az OpenSSL 3 Provider architektúráját használva tölti be ezeket az algoritmusokat az OpenSSL környezetbe. A Provider architektúra az OpenSSL 3 egyik legfontosabb újítása, amely lehetővé teszi külső kriptográfiai modulok dinamikus integrációját.

4.1.6. liboqs fordítása és telepítése

Első lépésként a liboqs forráskódját kell letölteni.

```
git clone --branch main https://github.com/open-quantum-safe/liboqs.git
cd liboqs
```

Ezután build könyvtár készül:

```
mkdir build
cd build
```

A projekt fordítása CMake és Ninja használatával történik.

```
cmake -GNinja ..
ninja
```

Telepítés:

```
sudo ninja install
sudo ldconfig
```

Az ldconfig parancs frissíti a dinamikus linker cache-t, így a rendszer felismeri az újonnan telepített library-kat.

4.1.7. OpenSSL 3.3 fordítása

A legtöbb Linux disztribúció saját OpenSSL verziót biztosít, azonban ezek általában nem kompatibilisek teljes mértékben az oqs-provider legfrissebb verzióival. Emiatt szükséges egy külön OpenSSL build létrehozása.

A forráskód letöltése:

```
git clone https://github.com/openssl/openssl.git
cd openssl
git checkout openssl-3.3
```

A konfiguráció során külön telepítési útvonal került megadásra.

```
./Configure \
--prefix=/opt/openssl-pqc \
linux-x86_64
```

A --prefix paraméter célja, hogy az új OpenSSL verzió ne írja felül a rendszer alapértelmezett könyvtárait.

Fordítás:

```
make -j$(nproc)
```

Telepítés:

```
sudo make install
```

4.1.8. OQS Provider fordítása

Az oqs-provider biztosítja a kapcsolatot az OpenSSL és a liboqs között.

```
git clone https://github.com/open-quantum-safe/oqs-provider.git
cd oqs-provider
```

Fordítás:

```
cmake -DOPENSSL_ROOT_DIR=/opt/openssl-pqc .
make
```

Telepítés:

```
sudo make install
```

4.1.9. Környezeti változók konfigurálása

```
export PATH=/opt/openssl-pqc/bin:$PATH
export LD_LIBRARY_PATH=/opt/openssl-pqc/lib64:$LD_LIBRARY_PATH
export OPENSSL_MODULES=/opt/openssl-pqc/lib64/openssl-modules
```

A labor során gyakori probléma volt a következő hiba:

```
error while loading shared libraries: libssl.so.4
```

Ez azt jelenti, hogy a dinamikus linker nem találta az új OpenSSL library fájlokat.

4.1.10. Az OpenSSL működésének ellenőrzése

```
openssl version -a
```

Provider modulok listázása:

```
openssl list -providers
```

PQC algoritmusok listázása:

```
openssl list -kem-algorithms
```

4.1.11. Mosquitto MQTT broker telepítése

A labor MQTT broker komponense az Eclipse Mosquitto lett. A Mosquitto egy könnyűsúlyú, nyílt forráskódú MQTT broker, amely széles körben elterjedt IoT környezetekben.

```
sudo apt install -y mosquitto mosquitto-clients
```

4.1.12. TLS tanúsítványok létrehozása

CA tanúsítvány létrehozása:

```
openssl req -x509 -new -nodes \
-keyout ca.key \
-out ca.crt \
-days 365
```

Szerverkulcs generálása:

```
openssl genrsa -out server.key 2048
```

CSR létrehozása:

```
openssl req -new \  
-key server.key \  
-out server.csr
```

Tanúsítvány aláírása:

```
openssl x509 -req \  
-in server.csr \  
-CA ca.crt \  
-CAkey ca.key \  
-CAcreateserial \  
-out server.crt \  
-days 365
```

4.1.13. Mosquitto TLS konfiguráció

```
listener 8883
```

```
cafile /etc/mosquitto/certs/ca.crt  
certfile /etc/mosquitto/certs/server.crt  
keyfile /etc/mosquitto/certs/server.key
```

```
require_certificate false
```

4.1.14. A broker indítása és tesztelése

```
sudo systemctl restart mosquitto  
sudo systemctl status mosquitto
```

TLS kapcsolat tesztelése:

```
openssl s_client \  
-provider default \  
-provider oqsprovider \  
-connect localhost:8883
```

4.1.15. MQTT kommunikáció ellenőrzése

Subscriber indítása:

```
mosquitto_sub \  
-h localhost \  
-p 8883 \  
--cafile ca.crt \  
-t test/topic
```

Publisher indítása:


```
mosquitto_pub \  
-h localhost \  
-p 8883 \  
--cafile ca.crt \  
-t test/topic \  
-m "Hello PQC"
```

4.1.16. Összegzés

A kialakított labor sikeresen demonstrálja, hogy az MQTT alapú IoT kommunikáció integrálható posztkvantum kriptográfiai algoritmusokkal. Az OpenSSL 3 Provider architektúrája és az Open Quantum Safe projekt lehetővé teszi modern PQC algoritmusok TLS környezetben történő használatát.

A labor előnye, hogy reprodukálható, izolált és könnyen bővíthető. A rendszer megfelelő alapot biztosít további kutatásokhoz, például TLS handshake mérésekhez, hálózati teljesítményvizsgálatokhoz vagy IoT környezetekben végzett kompatibilitási tesztekhez.

5. fejezet

Mérések

Jelen fejezet a kísérleti mérések végrehajtását és a nyers adatok gyűjtésének folyamatát írja le. A mérések célja a CRYSTALS-Kyber és a BIKE algoritmusok futási idejének, memóriaigényének és — ahol alkalmazható — energiafogyasztásának meghatározása a korábban bemutatott tesztkörnyezetben.

5.1. Mérési forgatókönyvek

5.1.1. Alap kriptográfiai műveletek mérése

5.1.2. TLS kézfogás (handshake) mérése

5.1.3. Terheléses mérés

5.2. Mérési módszer és adatgyűjtés

5.2.1. Időmérés megvalósítása

5.2.2. Memóriahasználat mérése

5.2.3. Energiafogyasztás mérése

5.3. Mérési eredmények

5.3.1. Kyber-512 eredmények

5.3.2. Kyber-768 és Kyber-1024 eredmények

5.3.3. BIKE-L1 eredmények

5.3.4. BIKE-L3 és BIKE-L5 eredmények

5.3.5. TLS kézfogás összesítő adatok

5.4. A mérések összefoglalása

6. fejezet

Eredmények kiértékelése

Jelen fejezet a mért adatok részletes elemzését és értelmezését tartalmazza. A fejezet célja, hogy a nyers mérési eredményekből következtetéseket vonjon le a CRYSTALS-Kyber és a BIKE algoritmusok IoT alkalmazhatóságáról, és összehasonlítsa a két protokoll teljesítményét különböző szempontok szerint.

6.1. Teljesítmény összehasonlítása

6.1.1. Futási idő összehasonlítása

6.1.2. Memóriaigény összehasonlítása

6.1.3. Kulcs- és kapszulaméret összehasonlítása

6.1.4. TLS kézfogás teljesítménye

6.2. Biztonsági szempontú értékelés

6.2.1. Biztonsági szintek és NIST kategóriák

6.2.2. Ismert kriptanalitikai eredmények

6.3. Alkalmazhatósági elemzés

6.3.1. Erőforrás-korlátozott eszközökre vonatkozó következtetések

6.3.2. Protokoll-szintű kihívások

6.3.3. Kyber és BIKE összehasonlító értékelése

6.4. Az eredmények összefoglalása

7. fejezet

Összefoglalás és javaslatok

Jelen fejezet összefoglalja a dolgozat főbb eredményeit, levon következtetéseket a posztkvantum-kriptográfiai algoritmusok IoT alkalmazhatóságáról, és javaslatokat fogalmaz meg mind a gyakorlati bevezetés, mind a jövőbeli kutatások irányára vonatkozóan.

7.1. A dolgozat eredményeinek összefoglalása

7.2. Javaslatok IoT rendszerek PQC migrációjához

7.3. Jövőbeli kutatási irányok

Köszönetnyilvánítás

Ez nem kötelező, akár törölhető is. Ha a szerző szükségét érzi, itt lehet köszönetet nyilvánítani azoknak, akik hozzájárultak munkájukkal ahhoz, hogy a hallgató a szakdolgozatban vagy diplomamunkában leírt feladatokat sikeresen elvégezze. A konzulensnek való köszönetnyilvánítás sem kötelező, a konzulensnek hivatalosan is dolga, hogy a hallgatót konzultálja.

Irodalomjegyzék

- [1] Frank Arute és mások: Quantum supremacy using a programmable superconducting processor. *Nature*, 574. évf. (2019), 505–510. p.
- [2] Kevin Ashton: That ‘internet of things’ thing. *RFID Journal*, 2009. Originally coined 1999.
- [3] Bike: Bit flipping key encapsulation. <https://bikesuite.org/>. Project page and submission materials, accessed 2026-05-26.
- [4] Crystals-kyber: Specification and supporting documentation. <https://pq-crystals.org/kyber/>. Project page and specification, accessed 2026-05-26.
- [5] Whitfield Diffie–Martin E. Hellman: New directions in cryptography. *IEEE Transactions on Information Theory*, 22. évf. (1976) 6. sz., 644–654. p.
- [6] Lov K. Grover: A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing* (konferenciaanyag). 1996, ACM, 212–219. p.
- [7] IoT Analytics: State of iot – spring 2023. <https://iot-analytics.com/number-connected-iot-devices/>, 2023.
- [8] Brian Krebs: Hacked cameras, dvrs powered today’s massive internet outage. Krebs on Security, 2016. <https://krebsonsecurity.com/2016/10/hacked-cameras-dvrs-powered-todays-massive-internet-outage/>.
- [9] McKinsey Global Institute: The internet of things: Mapping the value beyond the hype. McKinsey & Company report, 2015.
- [10] Nist selects first four quantum-resistant cryptographic algorithms. <https://www.nist.gov/news-events/news/2022/07/nist-selects-first-four-quantum-resistant-cryptographic-algorithms>. NIST press release, 2022.
- [11] Post-quantum cryptography project. <https://csrc.nist.gov/projects/post-quantum-cryptography>. NIST project page, accessed 2026-05-26.
- [12] Claude E. Shannon: Communication theory of secrecy systems. *Bell System Technical Journal*, 28. évf. (1949) 4. sz., 656–715. p.
- [13] Peter W. Shor: Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science* (konferenciaanyag). 1994, IEEE, 124–134. p.
- [14] Mark Weiser: The computer for the 21st century. *Scientific American*, 265. évf. (1991) 3. sz., 94–104. p.

Függelék

F.1. Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

- ☐ **Nem használtam** semmilyen generatív MI segédeszközt.
- ☐ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	Generatív MI eszköz(ök) neve	Érintett részek (fejezet, oldalszám, hivatkozás)	Használat becsült aránya (felhasználási módonként)
Irodalomkutatás			
Prompt lényegi része			
Programkód generálása			
Prompt lényegi része			
Új ötletek, megoldási javaslatok generálása			
Prompt lényegi része			
Vázlat létrehozása (szövegstruktúra, vázlatpontok)			
Prompt lényegi része			
Szövegblokkok létrehozása			
Prompt lényegi része			
Képek generálása illusztrációs célból			
Prompt lényegi része			
Adatvizualizáció, grafikonok generálása adatpontok alapján			
Prompt lényegi része			
Prezentáció készítése			
Prompt lényegi része			
Egyéb (nevezze meg)			
Prompt lényegi része			
Összesített százalékos érték (a feladat érdemi részére nézve):			
Összesített érték rövid, szöveges indoklása:			